

Разработка real-time интерфейса с поддержкой горизонтального масштабирования WebSocket-соединений для системы отслеживания задач

Защита выпускной квалификационной работы

Студент: Зайцев А. С., группа 41313

Руководитель: ст. преподаватель С. А. Рогачев

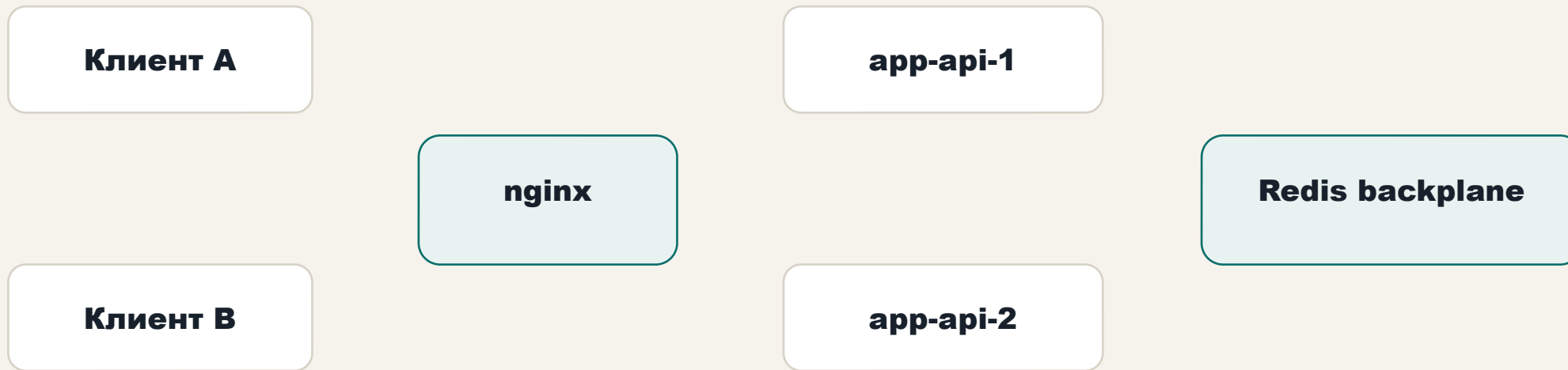
Санкт-Петербург, 2026

Проблема: single-node SignalR теряет события между узлами

Без межузлового канала
Каждый экземпляр знает только своих клиентов.
Событие, созданное на одном узле, не доходит до клиента на другом.

Цель работы
Восстановить корректную доставку событий между клиентами, подключенными к разным экземплярам backend-сервиса.

Целевая архитектура



SignalR-события публикуются между экземплярами через Redis; группы остаются локальным runtime-состоянием.

Что сделано

1 · Redis backplane для SignalR
Конфигурируется через
REDIS_CONNECTION_STRING

2 · Диагностика узла
SERVER_INSTANCE_ID, connectionId, machineName

3 · Стенд Docker Compose
app-api-1, app-api-2, Redis, PostgreSQL, nginx

4 · Node.js-клиент
Delivery, серия, rejoin, failover

Стенд и программа испытаний

Два режима

- С Redis backplane
- Без Redis backplane

Один и тот же код, разница — переменная окружения.

Четыре сценария

- Одиночная доставка
- Серия из 5 итераций
- Rejoin после разрыва
- Failover узла

Главный результат

**Без backplane
timeout 10000 мс
событие не доставлено**

**С backplane
ok: true - 5 из 5
p95 = 11,6 мс**

Один код, один сценарий — разница только в наличии межузлового канала.

Устойчивость: rejoin и failover

Rejoin после разрыва

Новый connectionId, повторный
JoinReportGroupAsync.

Событие после rejoin — 6,6 мс.

Failover узла

Клиент переподключился через nginx с app-api-2 на
app-api-1.

reconnect + rejoin — 240,3 мс.

Вывод

Ограничение single-node WebSocket-архитектуры устранено за счёт распределённого real-time контура.

Проблема
Без backplane событие
теряется между узлами.

Решение
SignalR + Redis backplane +
nginx + два app-api.

Доказательство
Delivery, серия, rejoin,
failover.